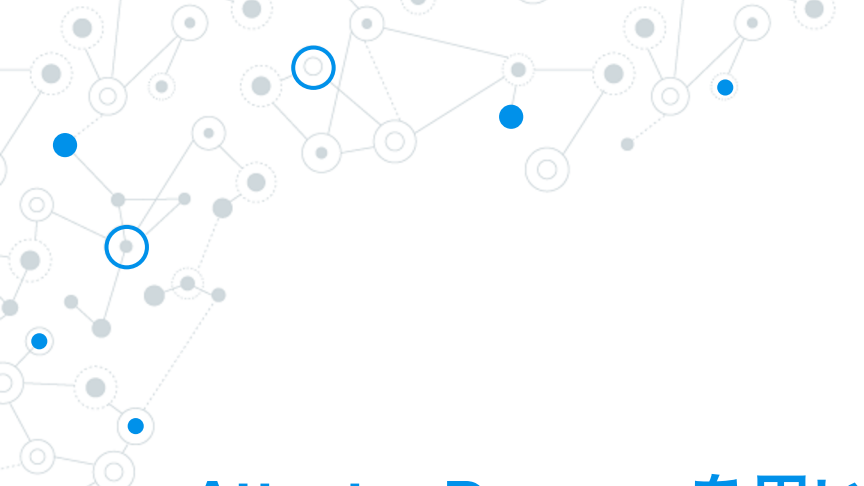




Attester Daemonを用いたアプリケーションの真正性検証の 安全性評価

佐々木怜名^{1,2} 石川裕^{2,3} 竹房あつ子^{2,4} 小口正人^{1,2}

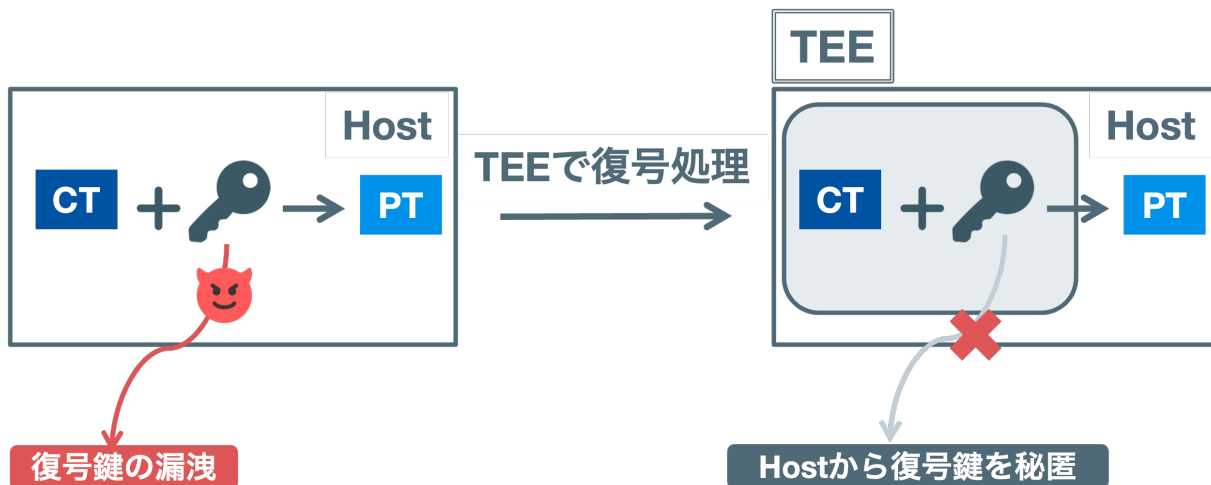
1. お茶の水女子大学 2. 情報・システム研究機構 セキュアコンピュータシステム研究開発センター
3. 大妻女子大学 4. 国立情報学研究所

本研究は、JST 経済安全保障重要技術育成プログラム【JPMJKP24U4】の支援を受けたものです。



■ TEEの暗号処理アプリケーションへの適用

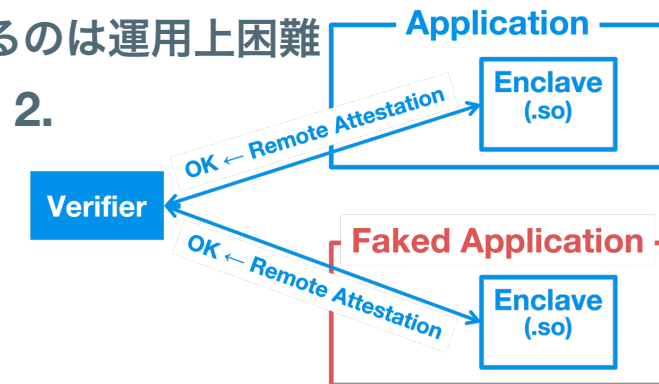
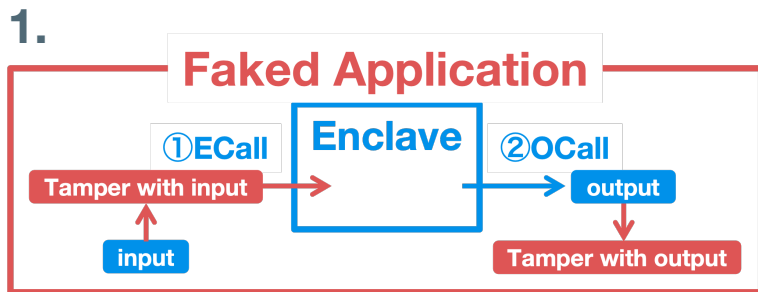
- 目的：
 - ホストOS侵害による復号鍵の漏洩を防ぎ、機密性保護の維持
 - 復号鍵の失効処理の効率化 (e.g. 属性ベース暗号) [1]
- 手法：TEE内で復号鍵を使う暗号処理を実行し、ホストから鍵を秘匿



TEEを用いたアプリケーションの課題

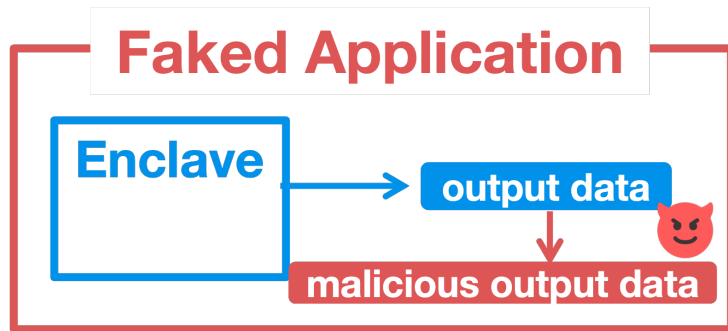
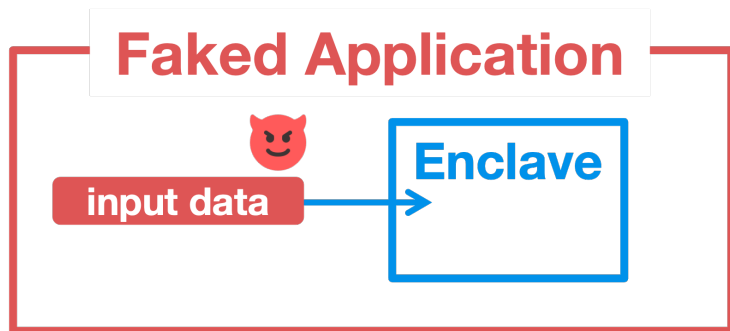
(Intel SGXが実現するTEEを対象とし、以降はSGXにおける隔離実行環境をEnclaveと呼ぶ)

1. EnclaveとHostの境界をまたぐデータは改ざん攻撃の対象となりうる^[2]
 - ECALLの引数やOCALLの戻り値が改ざんされる攻撃の対象となりやすい
2. Enclaveを呼び出すアプリケーション (Host) の真正性・完全性保証が困難
 - Enclaveのプログラム (.so形式) は可搬性があるが、任意のアプリケーションから呼び出され得る
 - 署名検証の対象となるアプリケーションが一意に定まらず、さらにそれらが依存する共有ライブラリを含めて、事前に署名・検証するのは運用上困難



アプリケーションの真正性を検証する必要性

- アプリが不正 or 改ざんされている場合，以下の可能性がある
 - 不正な引数によるEnclave内での暗号処理の実行
 - Enclave内での暗号処理結果が不正に利用
- ✓ Enclaveを呼び出すアプリケーションが真正であることを検証する仕組みが，システム全体のセキュリティ担保に不可欠



本研究の貢献

■ 目的

- Enclaveを利用するアプリケーションとその実行環境の真正性・完全性の保証
- 暗号アプリケーションで用いる復号鍵の更新・失効

■ 提案

Enclave呼出し元アプリケーションのアテストセッションプロトコル

■ 貢献

- ✓ 提案プロトコルの攻撃モデルと安全性要件の設定
- ✓ 形式検証による, 提案プロトコルの存在証明と, 攻撃モデル (Replay attack, 改ざん・偽造) に対する安全性証明
- ✓ Rollback attackを防ぐため, TPMモノトニックカウンタの利用可能性の検討に向けた, TPM2.0コマンドのマイクロベンチマークの測定

関連技術（TPM）

■ ハードウェアRoot of Trust

デバイス固有の鍵管理

- Endorsement Key (EK) : 製造時に埋め込まれた不変の公開鍵ペアで、チップの正当性を証明
- Attestation Key (AK) : EKを基に生成される署名用鍵, PCR (Platform Configuration Register) の外部証明に使用

■ 起動プロセスの整合性測定 (Measurement & PCR)

Chain of Trust: BIOS/UEFI → OSカーネルと順次ハッシュ値を計算・PCRに蓄積し、起動経路の改ざんを検知

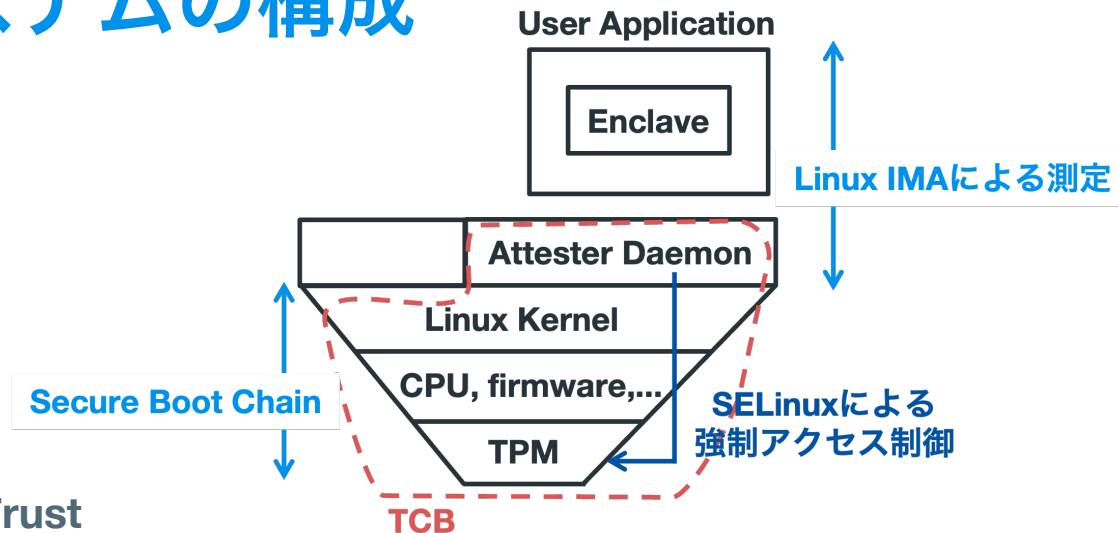
■ 機密データの保護 (Seal/Unseal)

特定のPCR状態に機密データを紐づけて暗号化・復号

■ 運用の堅牢性と鮮度 (freshness) の担保 (モノトニックカウンタ)

インクリメントのみ可能な耐タンパカウンタ

提案システムの構成



Root of Trust

- **TPM**:起動時・実行時の測定値蓄積, PCR保護, Quote
- **Intel SGX HW**: Enclaveの実行・Local attestation

TCB (Trusted Computing Base) の構成

ファームウェア, ブートローダ, Linux Kernel, Attester Daemon

- **Secure Boot Chain**により, ブートローダからLinux Kernelまでを測定
- **Attester Daemon**: 起動時にLinux IMAの測定
- **SELinux^[3]**により, TPMへのアクセス権限をAttester Daemonのみに限定

提案システムの概要

■ Intel SGXを用いる暗号アプリケーション

アプリケーションのアテストーション後に、Verifierを兼ねる鍵管理サーバ (KMS) がEnclaveに鍵を配布し、Enclaveで復号処理を実行

○ アテストーション

ある特定のプラットフォームについて信頼されたHW上で改ざんされずに正しく実行していることを証明

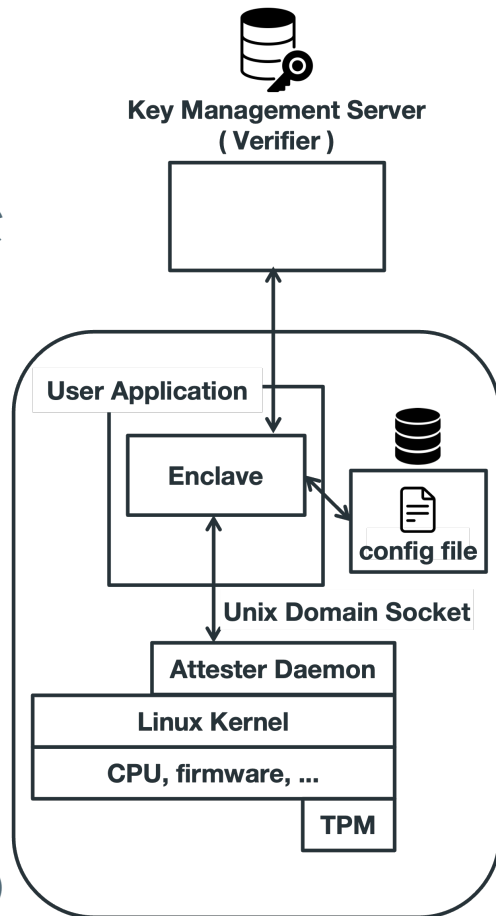
■ システムデプロイ

Secure bootとLinux IMAによるBoot chainからアプリケーションまでの測定 (measurement) と、TPMのPCRに測定値をextend

■ アプリケーションのアテストーション

Attester Daemonがエビデンスを生成

- KMS (Verifier)とEnclave間でRA-TLS通信のハンドシェイク時に、Enclaveのアテストーションとともに実施 (Remote Attestation: RA)
config fileにエビデンスや復号鍵などを保存、fileをEnclaveの固有の鍵でSealing
- ネットワーク遮断時は、App実行時にEnclaveがVerifierとなりLocal Attestation



提案プロトコルの概要

① システムデプロイ

Secure boot時のBoot chainとLinux IMAによる、ソフトウェアスタックからアプリケーションまでをmeasurement

② Attester Daemonのアテステーション

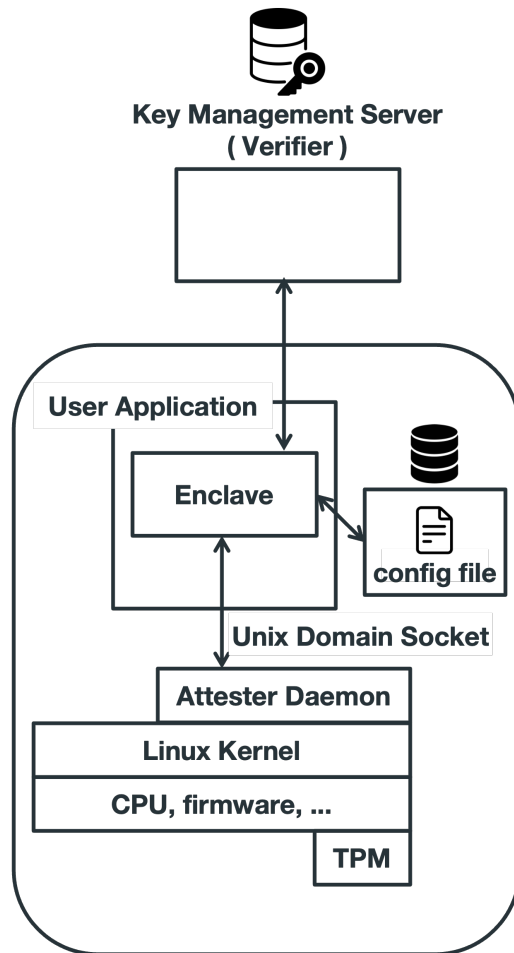
Attester DaemonがSecure bootしているか確認

③ Enclave・アプリケーションのアテステーション

アプリケーション実行時の測定値とSecure boot時のPCRを用いて検証し、アプリケーションが改ざんされていないことを確認

④ 暗号文の復号処理

鍵管理サーバ（KMS）から復号鍵を取得し、Enclave内で復号



起動時だけでなく実行時にもアプリケーションの真正性・完全性を保証

提案システムの攻撃モデル [1/3]

■ Replay Attack

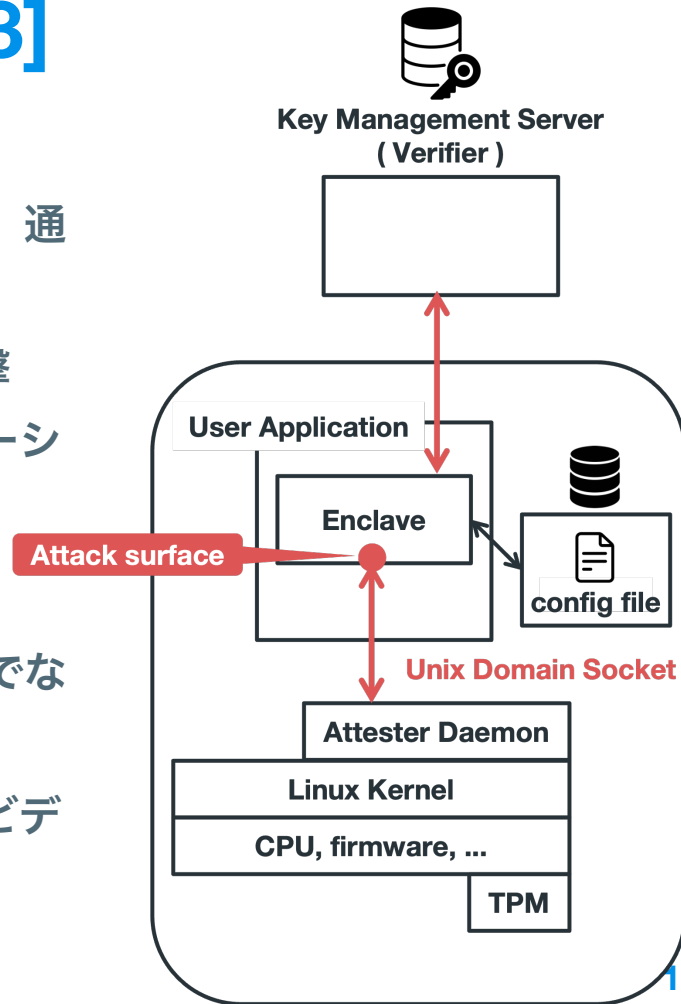
通信路でメッセージを盗聴・保存し、後に再送して、通信相手に正当なものと誤認させる攻撃

- Enclave↔Attester Daemon間の通信路への攻撃
- 過去のエビデンスを正当なものとしてアテストーション時に使用されるリスク

■ 改ざん・偽造

エビデンス中のデータを別のものに書き換え・正当でないエビデンスにすり替える攻撃

- アプリケーションのアテストーション前に、エビデンスに含まれるPCR値を改ざん・偽造



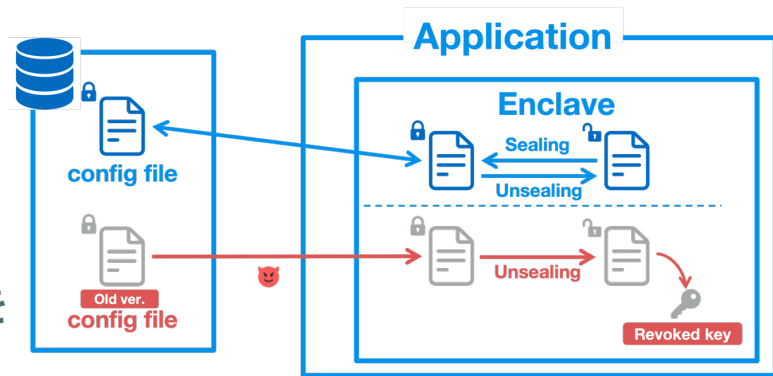
提案システムの攻撃モデル [2/3]

[3] Bryan Parno. Bootstrapping Trust in a "Trusted" Platform. In 3rd USENIX Workshop on Hot Topics in Security (HotSec 08), San Jose, CA, July 2008. USENIX Association.

Rollback Attack

攻撃者がシステムの正当な過去の状態を保存しておき、現在の最新状態をそれらで上書きして、不正な状態を正当だとシステムに誤認させる攻撃

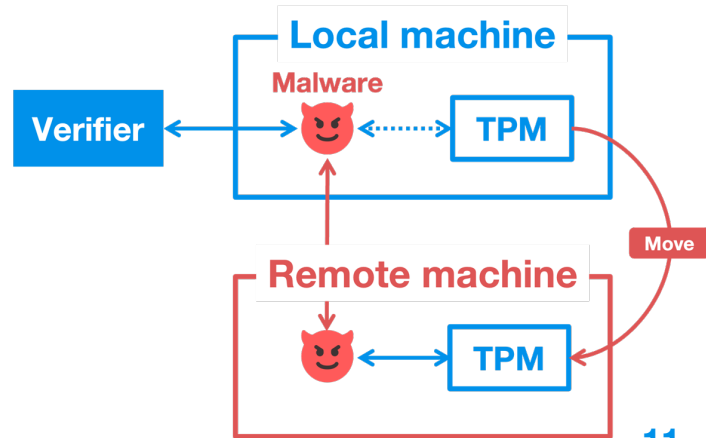
- 過去のconfig fileにあるエビデンスや失効鍵を正当なものとして使用するリスク



Cuckoo Attack^[3]

TPMの物理的挿し替え等により、攻撃者に制御される環境下にある正当なTPMへ中継されてしまう攻撃

- エビデンスの真正性をTPMの署名によって担保できないリスク



提案システムの攻撃モデル [3/3]

■ 各攻撃モデルに対応する安全性要件を設定

Security goal		Attack model
G1	Freshness of Evidence	Replay attack
G2	Integrity of Evidence	改ざん・偽造
G3	Freshness of Config file's state	Rollback attack
G4	Integrity of Config file's state	
G5	Integrity of Host code	Cuckoo attack
G6	Binding between Host Identity & Host code	
G7	Authenticity of Evidences	

攻撃モデルへの対処[1/2]

■ Replay attack, 改ざん・偽造

- KMSによるAppのRAリクエストと共にnonceを発行
- nonceを含めたエビデンス (TPM Quote) を生成・TPMによる署名
 - 署名検証でQuoteがTPMで作られたことを確認 (改ざん・偽造)
 - 埋め込まれたnonceが最新の要求時に発行した値と一致し, Quoteが最新ものであると確認 (Replay attack)

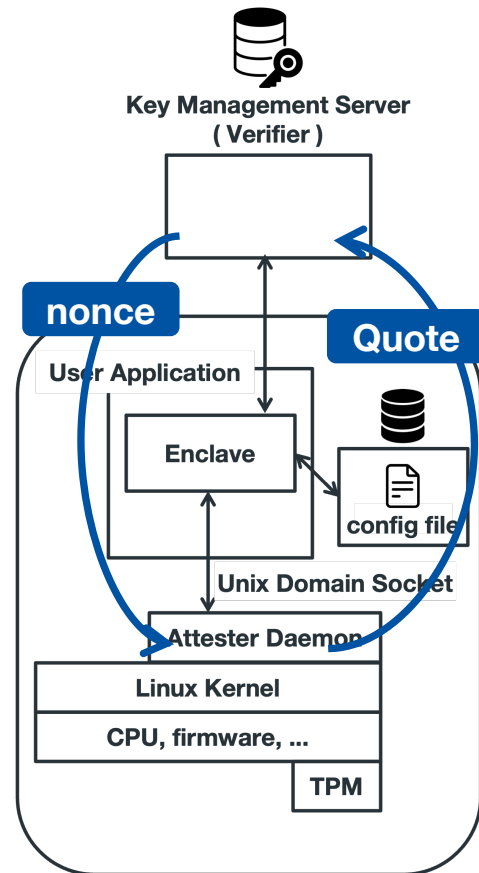
Unix domain socket通信路でもnonceのfreshnessを確認
(Replay attack)



形式検証

■ Cuckoo Attack

- Attester Daemonが取得するAppのPIDをハッシュ化しTPMで署名
 - 署名検証と, EnclaveによるApp PIDのハッシュの一致を確認
 - Faked Appが仲介している場合はPIDのハッシュは不一致



攻撃モデルへの対処 [2/2]

[4] S. Matetic et al., "ROTE: Rollback Protection for Trusted Execution," USENIX Security, 2017.

Rollback attack

- 鍵の失効やエビデンスの更新のたびに、TPMモノトニックカウンタの値をインクリメント
 - configファイル内に鍵やエビデンスと共にTPMカウンタフィールドを設けて格納
 - ファイルのカウンタ値がTPM内部の現在値より古い場合、攻撃の検知が可能

既存研究^[4,5]ではTPMのカウンタを利用する手法では以下の課題を指摘

- カウンタのインクリメント頻度の制限
 - TPM1.2では5sごとの更新を保証^[4]
- 処理速度の遅延
 - NVRAM write: 約100ms^[4]
- 提案システムでは暗号アプリケーションを想定
 - アプリケーションの新しいエビデンスの保存・復号鍵の更新/失効のときにインクリメント

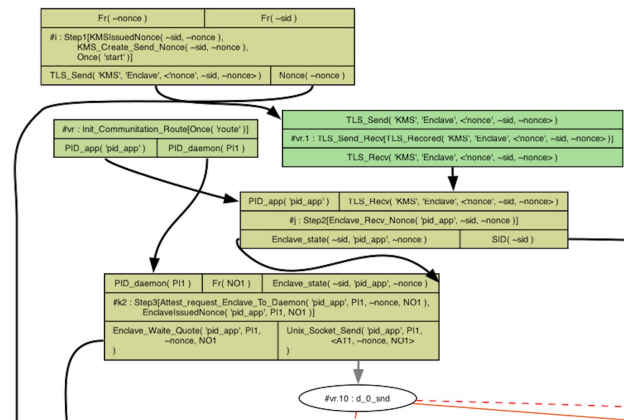


TPM2.0のコマンドレベルのマイクロベンチマークの測定

Tamarinを用いた形式検証

■ Tamarin prover: セキュリティプロトコルを形式検証するためのツール

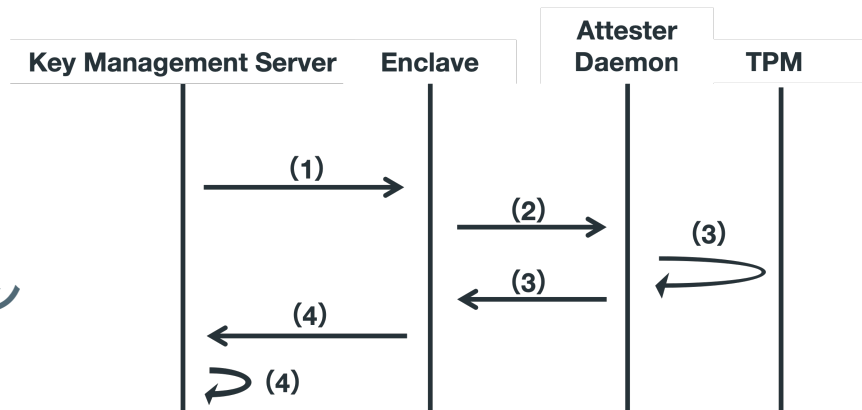
- プロトコルの各エンティティの動作, Attack model, 検証対象のセキュリティ特性を, 項書き換え規則・一階述語論理式により記述
- 満たすべき安全性の条件を論理式としてlemmaを記述
- セキュリティ特性を破る攻撃トレースが反例として提示
- 通信路の攻撃はDolev-Yaoモデル (改ざん・挿入・再送・破棄) ▼Traceの一部



提案プロトコルのモデル化

■ 検証対象プロトコル: ネットワーク接続時におけるアプリケーションのアテステーション

- (1) 鍵管理サーバからnonceを取得
- (2) エビデンス生成要求
- (3) エビデンス生成と返信
- (4) アプリケーションのアテステーション



■ プロトコルのモデル化

- 通信
 - Enclave-Daemon間のUnix domain socket通信は攻撃者に制御可能な通信
 - VerifierとEnclave間のTLS通信は状態遷移として簡素化
- アプリケーションのEnclave呼び出しは、Enclaveの状態遷移として抽象化
- EnclaveのRemote Attestationは先行研究^[5]より省略

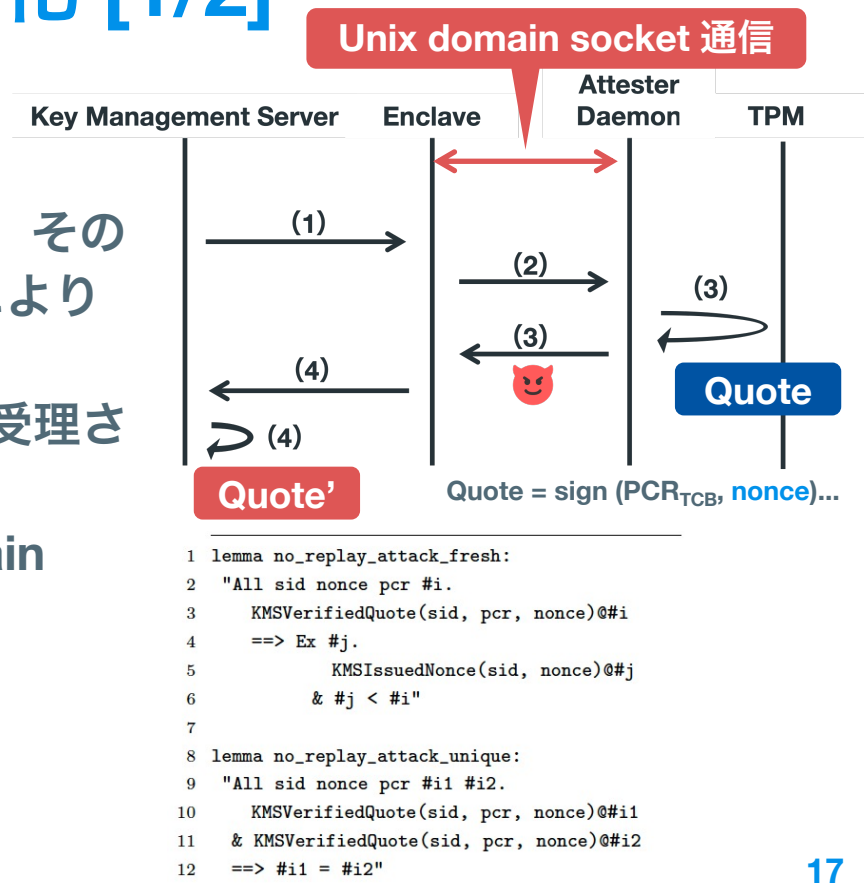
セキュリティ特性のモデル化 [1/2]

■ Replay attack

以下をlemmaとして記述

- VerifierがTPM Quoteを受理した時, そのQuoteに含まれるnonceがVerifierにより事前に発行されたものであること
- 同一のnonceを含むQuoteが複数回受理されないこと
- (Enclave-Daemon間のUnix domain socket通信に関して)も同様)

	Security goal	Attack model
G1	Freshness of Evidence	Replay attack
G2	Integrity of Evidence	改ざん・偽造



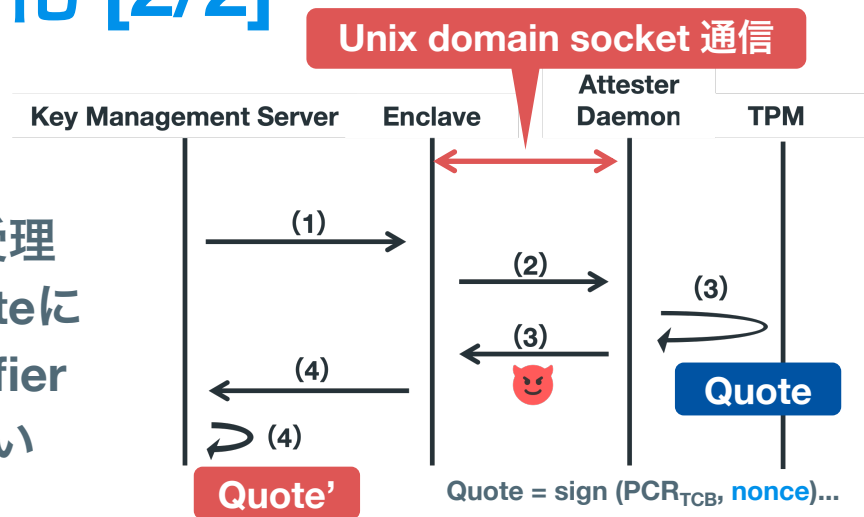
セキュリティ特性のモデル化 [2/2]

■ 改ざん・偽造

以下をlemmaとして記述

- VerifierがQuoteを検証成功として受理した場合，エビデンスの中身（Quoteに含まれるnonce・PCR_{TCB}）は，Verifierに保存されているQuoteの値と等しい

Security goal		Attack model
G1	Freshness of Evidence	Replay attack
G2	Integrity of Evidence	改ざん・偽造



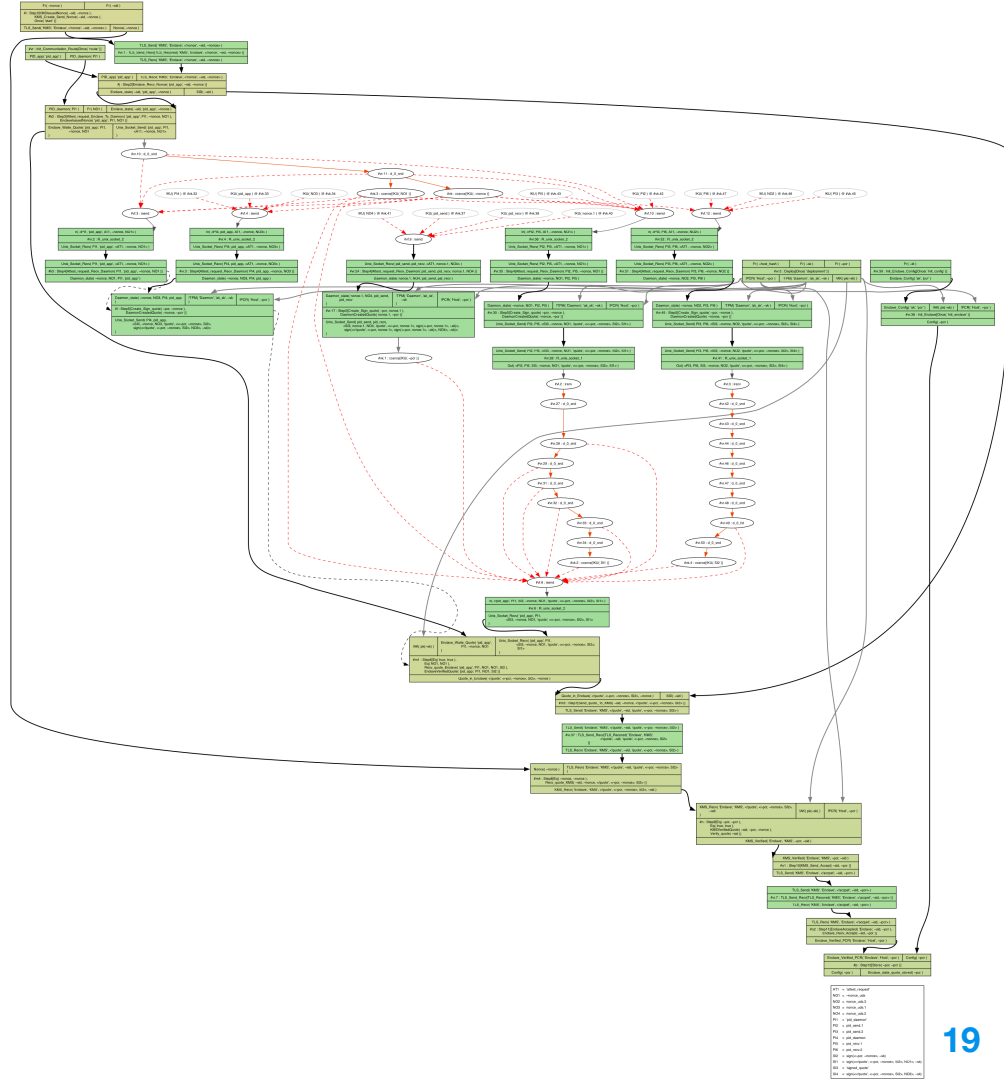
```

1 lemma integrity:
2   "All sid PCR_tcb nonce #i.
3     KMSVerifiedQuote(sid, PCR_tcb, nonce)@#i
4   ==> Ex #j.
5     DaemonCreatedQuote(nonce, PCR_tcb)@#j
6     & #j < #i"

```

検証結果

- 改ざん・偽造
エビデンスの完全性が担保されることを確認



TPM2.0コマンドレベルのマイクロベンチマーク

■ Rollback attackを防ぐため、カウンタとエビデンス/復号鍵を紐付けてconfig fileに保存

■ TPM2.0 ESAPIコマンド連続実行のE2E遅延を測定（100回）

- Esys_*()コマンドの呼び出し開始から終了までを測定
- ESAPI：対応するTPMコマンドを呼び出す上位API
- ESAPI, TCTI, OS・ドライバ処理, TPMとの通信時間なども含む

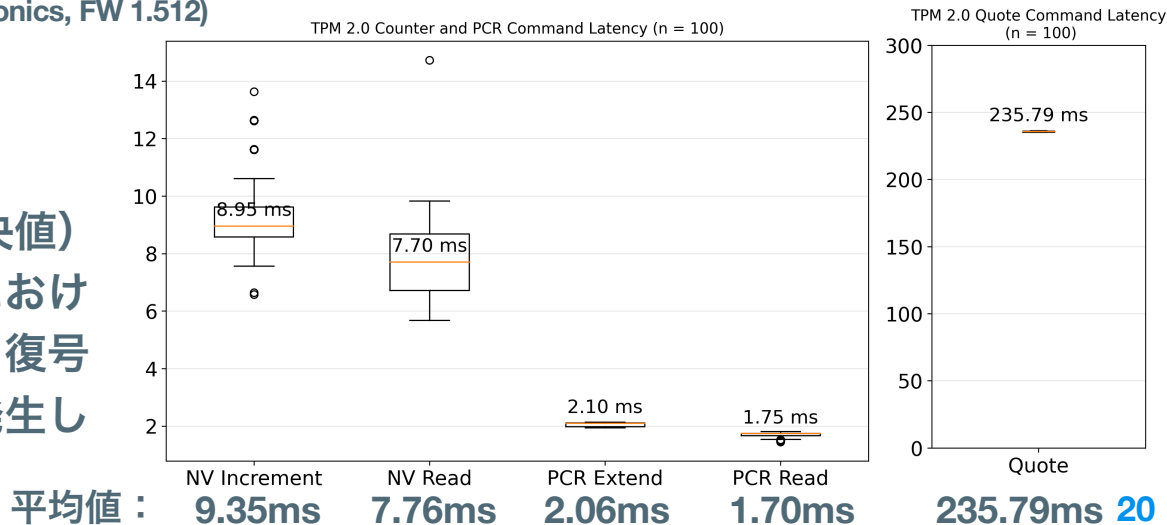
Platform: Intel Xeon Silver 4410Y, 128 GB RAM

TPM: HPE TPM 2.0 module (STMicroelectronics, FW 1.512)

Software: Ubuntu 24.04, tpm2-tss 4.2.0

■ カウンタインクリメント:

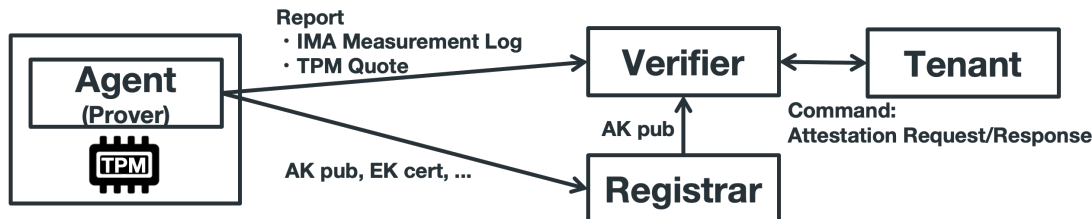
- 更新頻度: 107回/s
 - 処理遅延: 8.95 ms（中央値）
- 暗号アプリケーションにおける、エビデンスの更新・復号鍵の失効は、高頻度で発生しないため、許容範囲内



関連研究

■ Keylime^[6]

分散マシン環境下での、TPMを用いたAttestationと実行時の完全性測定



[6] <https://keylime.readthedocs.io/en/latest/index.html>
[7] M. Ruffin, C. Wang, G. Almasi, A. Adebayo, H. Franke and G. Wang, "Towards Continuous Integrity Attestation and Its Challenges in Practice: A Case Study of Keylime," 2025 55th Annual IEEE/IFIP International Conference on (DSN, Naples, Italy, 2025, pp. 256-265, doi: 10.1109/DSN64029.2025.00036.

■ Towards continuous integrity attestation and its challenges in practice: A case study of keylime.^[7]

- KeylimeやIMAを用いた継続的な完全性検証の課題の指摘
 - 予期しないOSのアップデートによるアテステーション時の攻撃の誤検知
 - 攻撃者がIMAの署名検証を透過する脆弱性が存在
- ネットワークがオンライン環境下での検証プロセスに依存

本研究

- DaemonがAttesterとなり、アプリケーション実行時に真正性を検証
- オフライン環境下でのLocal Attestation
- ローカルに保存されるエビデンスなどのロールバック耐性を保持

まとめと今後の課題

まとめ

- TEEを呼び出す実行主体であるアプリケーション・実行環境のアテステーション
プロトコルの攻撃モデルと安全性要件の設定
 - 形式検証：提案プロトコルの存在証明・Replay attack/改ざん/偽造に対する安全性証明
 - TPM2.0コマンドのマイクロベンチマーク
 - TPMカウンタを用いた耐ロールバック手法の実現可能性の確認

今後の課題

- ✓ 形式検証：Rollback attack, Cuckoo attackに対する安全性証明
- ✓ 実装と評価